Courses
Tutorials
Interview Prep
Sign In
nterview Questions Projects Widgets UI Components Design & Animations Forms & Gestures Navigation & Routing Dart Android Kotli

Integrating Maps and Geolocation Services Flutter

Last Updated : 8 Jul, 2024

In today's Android application, incorporating maps and geolocation services is essential. Whether it be finding the user's location, finding nearby travel places, or providing navigation, these features might really enhance the user experience and ease it down. Flutter, Google's UI toolkit for building natively compiled applications, makes it very easy and simple to integrate such functionalities using various plugins and packages.

In this article, we will be guided through the process of integrating maps and geolocation services in a Flutter application using the **'flutter_map'** package with **OpenStreetMap**, which is a free and open-source solution. We'll also use the '**geolocator**' package to access the device's location services and display the user's current location on the map.

Prerequisites

Before we start with map integration in Flutter, ensure that you have the following prerequisites:

- Basic understanding of Flutter and Dart programming language.
- Ensure that you have the Flutter development environment set up on your machine. If not, follow the Flutter Installation Guide.
- An emulator or a physical device for testing the application.

Dependencies and API Used

We'll use the following dependencies:

- **flutter_map**: A versatile Flutter package for displaying maps using OpenStreetMap.
- **latlong2**: A package for geographical calculations.
- **geolocator**: A package for accessing the device's location services.

Make sure to Add these above-mentioned dependencies to your pubspec.yaml file:

```
environment:
  sdk: ">=2.12.0 <4.0.0"

dependencies:
  flutter:
    sdk: flutter
  flutter_map: ^7.0.0
  latlong2: ^0.9.1
  geolocator: ^12.0.0
```

Run '**flutter pub get**' to install these packages.

Directory Structure

Your project directory should look like this:

```
maps_and_geolocation/
  └── lib/
      ├── main.dart
      └── map_screen.dart
```

Configuring the Project

Let's Start making this flutter project and configure it to use the '**flutter_map**' package .

1. Create a new Flutter project :

```
flutter create maps_and_geolocation
```

2. Update the pubspec.yaml file with the above mentioned dependencies .

Implementing Maps and Geolocation :

Step 1 : Import the required packages

After creating the project file , open the 'main.dart' file inside the 'lib/main.dart' and import required packages :

```dart
import 'package:flutter/material.dart';
import 'package:flutter_map/flutter_map.dart';
import 'package:latlong2/latlong.dart';
import 'package:geolocator/geolocator.dart';
```

Loading Playground...

Step 2 : Create a StatefulWidget for the map screen

We'll create a **StatefulWidget** to manage the state of our map and location:

```dart
class MapScreen extends StatefulWidget {
  @override
  _MapScreenState createState() => _MapScreenState();
}

class _MapScreenState extends State<MapScreen> {
  // Map and location-related code will go here
}
```

Loading Playground...

Step 3 : Initialize the MapController and user's location

Inside the '**_MapScreenState**' class, initialize the '**MapController**' and the user's current location:

```dart
late final MapController _mapController;
LatLng _currentLocation = LatLng(28.6139, 77.2090); // Initial center (New Delhi, India)

@override
```

```
void initState() {
  super.initState();
  _mapController = MapController();
  _getCurrentLocation(); // Get the user's current location on widget initialization
}
```

Step 4 : Get the user's current location

Implement the '**_getCurrentLocation**' method to get the user's current location using the **'geolocator'** package:

```
Future<void> _getCurrentLocation() async {
  try {
    Position position = await Geolocator.getCurrentPosition(
        desiredAccuracy: LocationAccuracy.high);
    setState(() {
      _currentLocation = LatLng(position.latitude, position.longitude);
    });
  } catch (e) {
    print('Error getting location: $e');
  }
}
```

Create the map UI

In the build method of the '**_MapScreenState**' class, create the UI for the map screen:

```
@override
Widget build(BuildContext context) {
  return Scaffold(
    appBar: AppBar(
      title: Text('OpenStreetMap'),
    ),
    body: FlutterMap(
      mapController: _mapController,
      options: MapOptions(
        center: _currentLocation,
        zoom: 13.0,
      ),
      children: [
        TileLayer(
          urlTemplate: "https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png",
          subdomains: ['a', 'b', 'c'],
        ),
        MarkerLayer(
          markers: [
            Marker(
              width: 80.0,
              height: 80.0,
              point: _currentLocation,
              builder: (ctx) => Icon(
                Icons.location_on,
                size: 48.0,
                color: Colors.red,
              ),
            ),
          ],
        ),
      ],
    ),
  );
}
```

This code creates a **Scaffold** with an AppBar and a FlutterMap widget. The FlutterMap widget uses the **MapController** to control the map's state and the **MapOptions** to set the initial center and zoom level.

The **TileLayer** renders the OpenStreetMap tiles, and the **MarkerLayer** adds a marker at the user's current location, represented by an icon.

Run the Application

Connect an Android device or start an Android emulator, and run the app using the following command in your terminal:

```
flutter run
```

Complete Code for the Application

We have 2 files that are important in this project **main.dart** and **map_screen.dart file** .

1. main.dart file

Here is the complete code for that file :

```dart
import 'package:flutter/material.dart';
import 'map_screen.dart';

void main() => runApp(MyApp());

class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'GeeksforGeeks Demo',
      theme: ThemeData(
        primarySwatch: Colors.green,
      ),
      home: MapScreen(),
    );
  }
}
```

2. map_screen.dart file

```dart
import 'package:flutter/material.dart';
import 'package:flutter_map/flutter_map.dart';
import 'package:latlong2/latlong.dart';
import 'package:geolocator/geolocator.dart';

class MapScreen extends StatefulWidget {
  @override
  _MapScreenState createState() => _MapScreenState();
}

class _MapScreenState extends State<MapScreen> {
  late final MapController _mapController;
  LatLng _center = LatLng(28.6139, 77.2090); // Initial center (New Delhi, India)

  @override
  void initState() {
    super.initState();
```

```
    _mapController = MapController();
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text('GeeksforGeeks Maps'),
        backgroundColor: Color(0xFF2F8D46), // GeeksforGeeks primary green color
      ),
      body: FlutterMap(
        mapController: _mapController,
        options: MapOptions(
          initialCenter: _center,
          initialZoom: 13.0,
        ),
        children: [
          TileLayer(
            urlTemplate: "https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png",
            subdomains: ['a', 'b', 'c'],
          ),
        ],
      ),
      floatingActionButton: FloatingActionButton(
        onPressed: _getCurrentLocation,
        tooltip: 'Get Location',
        backgroundColor: Color(0xFF2F8D46), // GeeksforGeeks primary green color
        child: Icon(Icons.my_location, color: Colors.white),
      ),
    );
  }

  Future<void> _getCurrentLocation() async {
    bool serviceEnabled;
    LocationPermission permission;

    // Test if location services are enabled.
    serviceEnabled = await Geolocator.isLocationServiceEnabled();
    if (!serviceEnabled) {
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(content: Text('Location services are disabled.')),
      );
      return;
    }

    permission = await Geolocator.checkPermission();
    if (permission == LocationPermission.denied) {
      permission = await Geolocator.requestPermission();
      if (permission == LocationPermission.denied) {
        ScaffoldMessenger.of(context).showSnackBar(
          SnackBar(content: Text('Location permissions are denied')),
        );
        return;
      }
    }

    if (permission == LocationPermission.deniedForever) {
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(
          content: Text(
              'Location permissions are permanently denied, we cannot request permissions.'),
        ),
      );
      return;
    }

    // When permissions are granted, get the position of the device.
    Position position = await Geolocator.getCurrentPosition();
    setState(() {
      _center = LatLng(position.latitude, position.longitude);
      _mapController.move(_center, 13.0);
    });
  }
}
```

and when we run the main.dart file the output will be the Map of New Delhi , that is the initial location

and when we click on the geolocation button , it will change to current location of the user (In this case to Ahmendabad)

Output:

Initial Map View
The app initially centers the map on New Delhi, India

User Location View
Upon clicking the floating action button, the map updates to the user location (simulated as Ahmedabad, India).

Conclusion

Integrating maps and geolocation services in a Flutter application can significantly enhance the functionality and user experience. By using the **flutter_map** package with **OpenStreetMap,** you can avoid the need for an API key and billing information, making it an excellent choice for small projects or those just getting started.

> ***Note :*** *Another free alternative is* ***Mapbox****, which offers a free tier and does not require card information for small-scale applications. You can use the* ***mapbox_gl*** *package for integrating Mapbox in Flutter.*

Comment J jatssah... 3

Explore

Basics

Key Widgets

UI Components

Design & Animations

Forms & Gestures

Navigation & Routing

Hardware Interaction

Gallery Access and Camera in Flutter 3 min read

Camera Access in Flutter 3 min read

Background local notifications in Flutter 6 min read

Restrict Landscape mode in Flutter 2 min read

Sample Flutter Apps

Advance Concepts

Courses

GeeksforGeeks
Sanchhaya Education Private Limited

Corporate & Communications Address:
A-143, 6th Floor, Sovereign Corporate Tower, Sector- 136, Noida, Uttar Pradesh (201305)

Registered Address:
K 061, Tower K, Gulshan Vivante Apartment, Sector 137, Noida, Gautam Buddh Nagar, Uttar Pradesh, 201305

GET IT ON Google Play Download on the App Store

Company
About Us
Legal
Privacy Policy
Contact Us
Advertise with us
GFG Corporate Solution
Training Program

Explore
POTD
Job-A-Thon
Blogs
Nation
Skill Up

Tutorials
Programming Languages
DSA
Web Technology
AI, ML & Data Science
DevOps
CS Core Subjects
Interview Preparation
Software and Tools

Courses
ML and Data Science
DSA and Placements
Web Development
Programming Languages
DevOps & Cloud
GATE
Trending Technologies

Videos
DSA
Python
Java
C++
Web Development
Data Science
CS Subjects

Preparation Corner
Interview Corner
Aptitude
Puzzles
GfG 160
System Design

@GeeksforGeeks, Sanchhaya Education Private Limited, All rights reserved